Ultimate Basic — C64 compiler

A modern BASIC-like language that compiles directly to 6502 machine code for the Commodore 64. Output: `.prg` files (VICE or real hardware) and `.d64` disk images.

Quick start

```
cargo build --release

# Compile to .prg (memory map printed on success)
ub build demo.ub -o demo.prg

# Verbose: also prints zero-page layout and hex dump
ub build demo.ub -v

# Compile + create .d64 disk image
ub build demo.ub --d64 disk.d64

# Compile + create .d64 with extra files embedded
ub build demo.ub --d64 disk.d64 --add music.prg --add loader.prg
```

Language reference

Variables and constants

```
var x = 10              # 8-bit integer (default)
var ptr: word = $0400   # 16-bit — two zero-page bytes, usable as 16-bit address
var f: float = 3.5      # Q8.8 fixed-point — hi byte = integer, lo byte =
fraction
var msg = "HELLO"       # string variable (pointer to inline PETSCII data)
var s: string = "TEXT"  # string with explicit type
var scores = array(10)  # byte array, 10 elements stored at $C000+
var times  = array_word(8) # word array, 8 word elements stored at $C000+
const BORDER_ADDR = $D020 # compile-time constant (substituted inline, no ZP slot)
```

Keywords and identifiers are **case-insensitive**: `PRINT`, `Print`, and `print` are all valid.

Type	Width	Notes
`int`	8-bit	default for numeric literals
`word`	16-bit	two ZP bytes; can be used as address in `poke`/`peek`
`float`	16-bit Q8.8	hi byte = integer part (0–255), lo byte = fractional part

Type	Width	Notes
`string`	pointer	ZP pair → null-terminated PETSCII in code segment
`array(N)`	N bytes	byte elements; lives at `$C000+`, not in ZP
`array_word(N)`	N×2 bytes	word (16-bit) elements; lives at `$C000+`, not in ZP

Comments

```
# hash comment
rem this is also a comment
var x = 5  # inline comment
var x = 5 : var y = 6  # colon separates statements on one line
```

Operators

```
x = x + 1
y = a * b - c / 2
z = x and 15             # bitwise AND
w = a or b               # bitwise OR
v = a xor b              # bitwise XOR
m = x shl 3              # shift left
n = x shr 2              # shift right
r = x mod 40             # 8-bit modulo (remainder); SEC/SBC/BCS loop
b = bnot x               # bitwise NOT: x XOR 255 (complement all 8 bits)
```

Comparisons: `==` `!=` `<` `>` `<=` `>=` (return 1/0)

Increment / Decrement

```
inc x                    # x = x + 1  (INC zp — single instruction)
dec x                    # x = x - 1  (DEC zp — single instruction)
```

For `word` variables carry is handled: `inc` uses `INC lo; BNE skip; INC hi`; `dec` uses `LDA lo; BNE skip; DEC hi; DEC lo`.

Compound Assignments

```
x += 5                   # x = x + 5
x -= 3                   # x = x - 3
x *= 2                   # x = x * 2
x /= 4                   # x = x / 4
x and= 15                # x = x and 15   (bitwise AND)
x or= 64                 # x = x or 64    (bitwise OR)
x xor= 255               # x = x xor 255  (bitwise XOR)
```

```
x shl= 2                    # x = x shl 2
x shr= 1                    # x = x shr 1
```

Print

```
print "HELLO"
print x
print x, y, "text"
print                        # blank line

print spc(5)                # print 5 space characters
print tab(20), "VALUE"    # move cursor to column 20, then print
print "A", spc(3), "B" # mix freely

print "A=" + a              # string + numeric var
print s1 + s2               # two string vars → sequential print
print "Hello " + "World" # two literals → compile-time fold
print chr$(13)              # print by PETSCII code
```

chr$

```
print chr$(65)              # output character with PETSCII code 65 ('A')
var c = chr$(n)             # store byte value n in variable c
print ">" + chr$(42)        # usable in string concat
```

Branching

```
if x == 1 then
  print "YES"
else
  print "NO"
end
```

Select / Case

```
select x
  case 1:
    print "ONE"
  case 2:
    print "TWO"
  else:
    print "OTHER"
end
```

`select expr` evaluates the expression once and compares it against each `case` value in order. The first matching case body is executed and control jumps to after `end`. The optional `else:` body runs if no case matches. All values must be 8-bit (0–255).

Loops

```
loop 5                  # counted loop (5 iterations)
  print "HI"
end

times 5                 # alias for loop N ... end
  print "HI"
end

loop                    # infinite loop
  x = x + 1
  if x == 5 then continue end  # skip to next iteration
  if x == 100 then break end
end

for i = 1 to 10         # for..next (preferred)
  if i == 5 then continue end  # skip to increment step
  print i
next

for i = 0 to 20 step 2
  print i
next i                  # variable name after 'next' is optional

loop i = 1 to 10        # legacy loop..end syntax — identical code
  print i
end

while x < 100
  x = x + 1
end

repeat                  # do-while: body runs at least once
  x = x + 1
until x == 100          # exits when condition is true
```

Labels and goto

```
label main_loop
  x = x + 1
  if x < 10 then goto main_loop end
```

Forward `goto` (label defined later) is fully supported.

`gosub label` / `return` jump to a label and return back (JSR / RTS at machine code level). The label must be a `label name` statement, not a `sub` — it has no parameters and shares the same zero-page scope. `gosub` supports forward references (label defined after `gosub`).

```
gosub draw_border
...
label draw_border
  # ... draw something ...
  return               # RTS — returns to the instruction after gosub
```

Subroutines

```
sub greet()
  print "HELLO!"
end

sub set_color(col)
  color border col
  color text   col
end

greet()              # call with parens
call greet           # call keyword (no parens)
set_color(6)
```

Parameters are passed via dedicated zero-page slots. No recursion (slots are static).

Arrays

```
var scores = array(8)    # 8 bytes at $C000

scores[0] = 100          # constant index → STA $C000
scores[i] = 99           # variable index → STA (ptr),Y
var v = scores[i]        # LDA (ptr),Y
print scores[2]          # usable inline in print

var times = array_word(8)  # 16 bytes (8×2) at $C000+

times[0] = $1234         # constant index → STA $C000 (lo), STA $C001 (hi)
times[i] = $5678         # variable index → ASL A for stride; (ptr),Y × 2
var t: word = times[1]   # LDA $C002, LDA $C003
```

16-bit (word) variables

```
var ptr: word = $0400     # two ZP bytes: lo=$00 hi=$04
poke ptr, 6               # STA (ptr),Y
var v = peek(ptr)         # LDA (ptr),Y
```

Bitmap graphics

```
graphics on               # VIC-II hires bitmap mode (320×200, 1bpp); bitmap at
$2000
graphics on multi         # VIC-II multicolor bitmap mode (160×200, 2bpp, 4
colours/cell)
graphics off              # return to text mode

gcls                      # clear bitmap (fills $2000-$3FFF) + set video matrix
colors

plot x, y                 # set pixel at (x, y);  x: 0-319,  y: 0-199
plot erase x, y           # clear pixel (AND ~mask)
plot xor x, y             # toggle pixel (EOR mask) — flicker-free animation
circle x, y, r            # midpoint circle centered at (x, y) with radius r; clips
off-screen points
line x1, y1, x2, y2       # Bresenham line from (x1,y1) to (x2,y2); x: 0-255, y: 0-
199
paint x, y                # 4-connected flood fill from (x, y); fills clear pixels
bounded by set ones
mplot x, y, color         # set multicolor pixel (x: 0-159, y: 0-199, color: 0-3);
requires graphics on multi
```

Both `graphics on` variants blank the display (`LDA $D011 / AND #$EF / STA $D011`) while switching VIC registers, then re-enable it in the target mode — prevents mode-switch glitches.

Block graphics (80×50)

```
graphics on block         # 80×50 block-pixel mode (text mode + custom 4-pixel
charset @ $2800)
graphics off              # return to text mode
gcls                      # clear block playfield: screen RAM $0400-$07FF + color
RAM $D800-$DBFF

plot4 x, y                # set block pixel at (x, y);  x: 0-79, y: 0-49
plot4 erase x, y          # clear block pixel at (x, y)
circle4 x, y, r           # draw midpoint circle in block pixels; clips to 80×50
```

A chunky low-res mode layered on standard 40×25 text. A 16-character custom charset copied to `$2800` encodes a 2×2 quadrant grid per character (bit3=TL, bit2=TR, bit1=BL, bit0=BR), so each text cell holds 2×2 block pixels → an effective 80×50 grid. No bitmap RAM is used (`$2000-$3FFF` stays free), making it faster than hires bitmap. `plot4` OR's the quadrant bit into the cell so overlapping pixels accumulate; `plot4 erase` clears it. `circle4` uses the same block-pixel helper to draw an outline circle in the 80×50 coordinate space. `gcls` clears both screen and color RAM. See `examples/block_demo.ub`.

Screen and color

```
cls                         # clear screen (KERNAL $E544)
cls fast                    # fast fill: screen RAM + color RAM + HOME

color text 14               # text color register $0286
color border 6              # $D020
color bg 0                  # $D021

screen 0, 0, 65             # write char 65 ('A') to screen RAM at col 0, row 0
($0400)
screen 10, 5, ch            # col 10, row 5 — col/row can be variables
screen 5, 3, 42, 7          # char 42 at col 5, row 3, color 7 (writes color RAM
$D800 too)
screen x, y, ch, col        # all four arguments as variables

display on                  # re-enable VIC display ($D011 DEN bit)
display off                 # blank display

lowercase                   # CHR$(14) → switch VIC-II to lowercase/uppercase charset
uppercase                   # CHR$(142) → switch VIC-II back to uppercase/graphics
charset

cursor 20, 10               # move cursor to col 20, row 10 (KERNAL PLOT $FFF0)
cursor x, y                 # column from variable x (0-39), row from y (0-24)

print at 20, 10, "HELLO" # cursor(20,10) + print in one statement (no trailing
newline)
print at x, y, "Score:", score  # any mix of exprs
print at 0, 0               # position only (no text, no newline)

scroll x 3                  # set horizontal fine scroll: $D016 bits 0-2 = 3 (0-7)
scroll y 2                  # set vertical fine scroll:   $D011 bits 0-2 = 2 (0-7)
scroll x n                  # value can be a variable or expression (masked to bits
0-2)
scroll x 7 narrow           # set fine scroll and force 38-column mode (hide edge
column)
scroll x 0 wide             # set fine scroll and restore 40-column mode
scroll row 12 left          # shift screen RAM row 12 left by one character
```

`lowercase` emits `LDA #$0E; JSR $FFD2` at runtime. String literals compiled after `lowercase` have their case swapped automatically — uppercase source chars are stored in the PETSCII lowercase slot (`$61+`) and lowercase source chars in the uppercase slot (`$41+`) — so `"Hello World"` source displays as **Hello World** on screen. `uppercase` emits `LDA #$8E; JSR $FFD2` and reverts to direct mapping. `cls` does **not** reset the charset mode.

`scroll x n` writes `(n AND 7)` into bits 0-2 of `$D016` (preserving bits 3-7). `scroll x n narrow` writes the fine-scroll bits and clears `$D016` bit 3 (38-column mode). `scroll x n wide` writes the fine-scroll bits and sets `$D016` bit 3 (40-column mode). `scroll y n` writes `(n AND 7)` into bits 0-2 of `$D011` (preserving bits 3-7). `scroll row R left` shifts one constant screen row left; write the new rightmost character with `screen 39, R, ch`. Useful for smooth hardware scrolling: decrement each frame from 7 down to 0, shift screen RAM, reset to 7.

`screen col, row, char [, color]` writes directly to screen RAM (`$0400 + row*40 + col`) and optionally to color RAM (`$D800 + row*40 + col`). Constant col/row: address computed at compile time.

Ultimate 64 — CPU Speed

```
speed 4              # set CPU to 4 MHz  (reads $D031, updates bits 0-3, writes
back)
speed 48             # 48 MHz  (maximum speed on U64)
speed max            # same as speed 48  (alias)
speed off            # back to 1 MHz  (alias for speed 1)

badlines on          # enable badline timing  ($D031 bit 7 = 0, default C64
behaviour)
badlines off         # disable badline timing ($D031 bit 7 = 1, more CPU cycles)

var t = turbo()      # 1 if turbo is active (bits 0-3 of $D031 != 0), 0 if at 1
MHz
```

Available MHz values: `1, 2, 3, 4, 5, 6, 8, 10, 12, 14, 16, 20, 24, 32, 40, 48`. Constant values are rounded down to the nearest available speed at compile time. Variable values are treated as a raw speed index (0–15) and OR'd into bits 0-3 of `$D031`.

$D031 index	MHz (U64)	MHz (U64 Elite-II)
0	1	1
3	4	4
6	8	10
11	20	24
15	48	64

Requires **U64 Turbo Control** set to `U64 Turbo Registers` or `Turbo Enable Bit` in the U64 config menu. On a regular C64 or emulator without the register, the `poke` to `$D031` is silently ignored.

Keyboard

```
var key = getch()         # busy-wait on $FFE4 until key; returns PETSCII code
var k   = inkey()         # non-blocking: returns PETSCII code, or 0 if no key
pressed
waitkey                   # wait until any key is pressed (CIA1 matrix scan; works
```

```
  during IRQ)
  var k   = waitkey()       # same but returns raw $DC01 column bits (0 bit = key
  pressed)
  var j = joy(2)            # read joystick port 2; returns inverted bits 0-4
  var j = joy(1)            # read joystick port 1
                            # bit0=up(1) bit1=down(2) bit2=left(4) bit3=right(8)
  bit4=fire(16)
  var mx = mouse_x()        # 1351 mouse X: SID POT X ($D419), 0-255
  var my = mouse_y()        # 1351 mouse Y: SID POT Y ($D41A), 0-255
  var mb = mouse_btn()      # mouse buttons: bit0=left (fire), bit1=right
```

Exit

```
bye                         # JSR $E544 (clear screen), clear STOP flag, RTS to BASIC
exit                        # alias for bye
```

Timing

```
wait 50                     # wait 50 raster-line transitions (~3.2 ms)
wait raster 100             # spin until $D012 == 100 (raster-split effects)
delay 1                     # wait 1 PAL frame (1/50 s ≈ 20 ms)
delay 20                    # wait 20 frames ≈ 0.4 s; n can be a variable (0-255)
```

`delay N` counts N complete PAL frames using raster line 200 as the frame boundary.

SID Sound

```
sound 0, $1CAD, 25          # voice 0, freq $1CAD (≈ middle C PAL), 25 frames
duration
sound 1, freq_word, 50      # voice 1, freq from word var, 50 frames (1 s at 50 Hz)
sound 2, 0, 0               # voice 2, silence

sid volume 15               # master volume full ($D418 = $0F); range 0-15
sid volume 0                # silence (master volume = 0)
sid stop                    # zero all 25 SID registers ($D400-$D418) — complete
silence
```

`sound <channel>, <freq>, <duration>` — duration in PAL frames (1/50 s each). Fixed ADSR: attack/decay `$09`, sustain/release `$F0`, sawtooth waveform. Master volume `$D418` always set to `$0F`.

`sid volume N` writes N to `$D418`. Bits 0-3 = volume (0-15), bits 4-7 = filter mode. `sid stop` emits a 10-byte zero-fill loop — faster than 25 individual pokes.

Music playback

`music play/stop/pause/resume` is a high-level alternative to the manual `sys sid_init` / `cia_timer` setup. Requires a prior `load sid` statement (defines `sid_init` / `sid_play`).

```
load sid "tune.sid"          # embed SID file (defines sid_init / sid_play)

music play                   # initialise sub-tune 0 + start CIA1 50 Hz IRQ
music play 1                 # start from sub-tune 1 (song number 0-based)
music stop                   # stop playback + zero all 25 SID registers
($D400-$D418)
music pause                  # disable CIA1 timer A IRQ (music freezes, SID
unchanged)
music resume                 # re-enable CIA1 timer A IRQ (continues from pause
point)
```

Statement	Effect
`music play [n]`	call `sid_init(n)`, configure CIA1 timer A at 19 656 cycles (~50 Hz PAL), install IRQ wrapper
`music stop`	disable CIA1 IRQ + zero all 25 SID registers
`music pause`	disable CIA1 IRQ (SID output remains frozen)
`music resume`	re-enable CIA1 IRQ (continues from pause point)

The IRQ wrapper (emitted once at the end of the program) does: ACK CIA1 timer A → `JSR sid_play` → `JMP $EA81`.

Sprites

```
sprite 0, x, y, $2000    # sprite 0: set X, Y position and data pointer
sprite 0, x, y           # without data pointer (keeps existing)
sprite on  0             # enable sprite 0 ($D015 |= bit0)
sprite off 0             # disable sprite 0 ($D015 &= ~bit0)
sprite color 0, 7        # sprite 0 color = yellow ($D027)
sprite multicolor 0, on  # enable multicolor mode for sprite 0 ($D01C |= bit0)
sprite multicolor 0, off # disable multicolor mode ($D01C &= ~bit0)
sprite expand x 0, on    # double width ($D01D |= bit0)
sprite expand x 0, off   # normal width ($D01D &= ~bit0)
sprite expand y 0, on    # double height ($D017 |= bit0)
sprite expand y 0, off   # normal height ($D017 &= ~bit0)
sprite priority 0, on    # behind background ($D01B |= bit0)
sprite priority 0, off   # in front of background ($D01B &= ~bit0)
var h = sprite_hit()     # sprite-sprite collision ($D01E, cleared on read)
var b = sprite_bg_hit()  # sprite-background collision ($D01F, cleared on read)
var x = sprite_x(0)      # read sprite 0 X position (lo byte, $D000)
var y = sprite_y(0)      # read sprite 0 Y position ($D001)
sprite_frame 0, $2000    # set sprite 0 data pointer only: $07F8+id = $2000>>6
                         # (does NOT change X/Y position)
```

X supports full 9-bit range (0–319): use a `word` variable for runtime values > 255. Sprite data pointer: `data_addr` must be 64-byte aligned; stored as `addr >> 6` at `$07F8+id`.

Sprite definition

```
sprdef 0
  $00,$3C,$00,  $00,$FF,$00,  $03,$FF,$C0,  $07,$FF,$E0,
  $0F,$FF,$F0,  $0F,$FF,$F0,  $1F,$FF,$F8,  $1F,$FF,$F8,
  $1F,$FF,$F8,  $0F,$FF,$F0,  $0F,$FF,$F0,  $07,$FF,$E0,
  $03,$FF,$C0,  $00,$FF,$00,  $00,$3C,$00,  $00,$00,$00,
  $00,$00,$00,  $00,$00,$00,  $00,$00,$00,  $00,$00,$00,
  $00,$00,$00
end
```

`sprdef id ... end` embeds 63 sprite bytes at the next 64-byte-aligned address in the code segment, emits a `JMP` over them, and automatically sets `$07F8+id = data_addr >> 6`. To use the same shape for multiple sprites, read back the pointer:

```
var pg = peek($07F8)   # pointer set by sprdef 0
poke $07F9, pg         # copy to sprites 1-7
```

Custom charset

```
charset $3800            # set base address for chardef (default $3800)

chardef 65               # redefine character 65 ('A')
  $18,$3C,$66,$7E,$66,$66,$66,$00
end

chardef 66               # fewer than 8 bytes are zero-padded
  $7C,$66,$7C,$66,$7C
end
```

`charset base` sets the destination address used by all subsequent `chardef` statements. `chardef id ... end` embeds 8 bytes inline in the code segment (preceded by a `JMP` to skip over them), then copies them to `charset_base + id*8` at runtime. Values must be compile-time constants; use `%` for binary literals (`%00011000`).

To activate a custom charset in VIC-II, set the character generator address via `$D018`:

```
charset $3800
chardef 1  $FF,$81,$81,$81,$81,$81,$81,$FF  end  # box border
poke $D018, $1A     # screen at $0400, charset at $3800 (bank 0)
```

Memory

```
poke $D020, 2            # STA $D020
poke addr_var, 6         # STA (addr_var),Y  — if addr_var is word type
var v = peek($D012)      # LDA $D012
var v = peek(addr_var)   # LDA (addr_var),Y  — if addr_var is word type

var w: word = peek16($C000)   # read 16-bit little-endian: lo=$C000, hi=$C001
poke16 $0314, $EA81           # write 16-bit little-endian: lo→$0314, hi→$0315
poke16 ptr, w                 # word var as address; word var as value
```

`peek16(addr)` reads two consecutive bytes (lo, hi) as a `word`. `poke16` writes lo then hi.

Disk I/O

```
load "PROGRAM"           # KERNAL LOAD: loads file from device 8 to its native
address
load "DATA", $C000       # loads file to a specific address
load "DATA", ptr         # addr from word variable

save "DATA", $C000, 4096 # KERNAL SAVE from $C000, 4096 bytes → device 8
save "PROG", start, len  # addr and len from word/int variables
```

`load` calls KERNAL `SETNAM`+`SETLFS`+`LOAD` (`$FFBD`/`$FFBA`/`$FFD5`). Without address: secondary address 0 (file's own 2-byte header used as load address). With address: secondary address 1 (file loaded to specified location). `save` calls `SETNAM`+`SETLFS`+`SAVE` (`$FFBD`/`$FFBA`/`$FFD8`). Requires both `addr` and `len`.

SID Music

```
load sid "tune.sid"           # embed SID music at its native load address
load sid "tune.sid", $2000    # override: embed at $2000 regardless of SID header
```

`load sid` reads a PSID or RSID file at **compile time**, strips the header, and appends the raw music bytes to the output `.prg`. After `load sid`, two compile-time constants become available:

Constant	Description
`sid_init`	Init routine address — call once with A = song number (0-based)
`sid_play`	Play routine address — call every frame (50 Hz PAL) from an IRQ handler

Both constants work anywhere a constant address is accepted: `sys`, `irq`, `poke`, expressions.

Typical usage:

```
load sid "music.sid"

sub music_irq()
  poke $D019, $FF         # ACK VIC raster IRQ
  sys sid_play            # advance one frame of music
  irq_exit                # JMP $EA81: restore A/X/Y + RTI (proper IRQ exit)
end

sys sid_init, 0           # initialise SID chip: A=0 → first sub-tune
irq music_irq, $C0        # raster IRQ at line $C0 → 50 Hz on PAL

sid volume 15             # master volume on
```

Notes:

- SID data is placed **after** all generated code, padded with zeros up to the load address. The compiler warns if the SID load address would overlap generated code.
- PSID v1 and v2 are supported. If the SID header's load address is 0, the first two data bytes are used as the address (PRG-style, little-endian).
- Only one `load sid` per program is meaningful (the last one wins).

Serial channel file I/O

```
open 1, 8, 2, "MYFILE"  # open logical file 1, device 8, secondary 2, name
"MYFILE"
open 2, 4, 7             # open printer (device 4), no filename
open ch, dev, sec        # channel, device, secondary from variables

print# 1, "HELLO"        # send "HELLO"+CR to logical file 1
print# ch, x, "text"     # any mix of vars, strings — same as print but to file

close 1                  # close logical file 1
close ch                 # channel from variable
```

`open` calls `SETNAM` ($FFBD) + `SETLFS` ($FFBA) + `OPEN` ($FFC0). Without a filename, SETNAM is called with length 0. `print#` routes output via `CHKOUT` ($FFC9), CHROUT per char (+ trailing CR), then `CLRCHN` ($FFCC). `close` puts the channel number in A and calls `CLOSE` ($FFC3).

Input

```
input score              # read up to 3 digits from keyboard → 8-bit int var
input "Name: ", name     # optional prompt string, then read line → string var
input "Score: ", score   # prompt + int input
```

`input` uses KERNAL BASIN (`$FFCF`) for blocking, echoed line input with DEL support.

- **Int var**: accepts only `0`–`9`, max 3 chars; converts to 8-bit value on CR.
- **String var**: accepts up to 30 chars; stores as null-terminated string; ZP pair updated.

Float / Fixed-Point

`float` variables use Q8.8 fixed-point format: the high byte is the integer part (0–255) and the low byte is the fractional part (0/256 ... 255/256).

```
var f: float = 3.5        # 3.5 → hi=3, lo=128 (= 0x0380)
var g: float = 0          # integer 0 is promoted to 0.0 automatically

f = 1.5                   # Q8.8 literal assignment
f = f + 1.5               # 16-bit Q8.8 arithmetic (result: 3.0)
f = f + g                 # float + float

var n = int(f)            # extract integer part (hi byte) → 8-bit int
print f                   # prints as "N.DD" (e.g. 3.5 → "3.50", 1.25 → "1.25")
```

Operation	Example	Notes
Literal	`3.5`, `0.25`, `1.0`	parsed as Q8.8 at compile time
Integer promotion	`f = 5`	stores 5.0 (hi=5, lo=0)
Add/sub	`f + 1.5`, `f - g`	16-bit Q8.8 arithmetic
Extract int	`int(f)`	returns hi byte as 8-bit int
Print	`print f`	format "N.DD", always 2 fractional digits

Caveat: Arithmetic overflow wraps at 255.255 (no saturation). Multiplication and division of two float variables are not yet supported — use `int()` + integer arithmetic for those cases.

Math functions

```
var a = abs(x - 20)       # two's-complement absolute value
var b = min(x, 39)        # 8-bit minimum
var c = max(x, 0)         # 8-bit maximum
var s = sgn(score)        # 0 = zero, 1 = positive (1-127), $FF = negative (128-
255)
var r = rnd()             # LCG pseudo-random 0-255; seed from raster line
var r = rnd(10)           # LCG pseudo-random 0-9 (rnd() mod n; result 0..n-1)
var s = sin(angle)        # sine: angle 0-255 (full circle), returns 0-255
(center=128)
var c = cos(angle)        # cosine = sin(angle+64)
var v = clamp(x, 0, 39)   # 8-bit unsigned clamp: result = max(lo, min(hi, val))

print hex(n)              # print as 2-digit uppercase hex
print bin(n)              # print as 8-bit binary string
```

String functions

```
var n = len(msg)        # length of null-terminated string var (0–255)
var c = asc(msg)        # PETSCII code of first character (0 if empty)
var c = asc("A")        # compile-time: constant PETSCII code
var n = val(s)          # runtime: parse decimal PETSCII string → 8-bit int (e.g.
"042" → 42)
var c = msg[i]          # string character at index i: PETSCII code of msg[i]
msg[i] = c              # write PETSCII byte c to string at index i — STA (ptr),Y
msg[0] = 72             # constant index → LDY #0; STA (ptr),Y
```

Number formatting

```
print hex(n)            # print as 2-digit uppercase hex
print bin(n)            # print as 8-bit binary string
print dec(n, 4)         # right-justified decimal in a field of 4 chars (e.g. 42
→ "  42")
print dec(n, width)     # width can also be a variable
```

`dec(n, width)` pads the number on the left with spaces to fill `width` characters. If the number has more digits than `width`, it is printed without padding (no truncation). In non-print contexts `dec(n, w)` evaluates to `n` unchanged (same as `hex`/`bin`).

REU (RAM Expansion Unit)

```
var ok = reu_present()   # 1 if REU detected, 0 if not (write/read test on $DF04)
var ok = reudet()        # alias for reu_present()

reu stash c64addr, bank, reu_addr, len  # copy C64 → REU
reu fetch c64addr, bank, reu_addr, len  # copy REU → C64
reu swap  c64addr, bank, reu_addr, len  # swap between C64 and REU
```

`reu_present()` performs a write/read-back test on REU register `$DF04`. Without an REU the write is lost (open bus), so the read-back differs — reliably detects presence without touching any side-effecting command register.

Parameter	Width	Notes
`c64addr`	16-bit	C64 RAM start — constant, `word` var, or 8-bit expr
`bank`	8-bit	REU bank number (0–7 for a 512 KB unit)
`reu_addr`	16-bit	Offset within the REU bank
`len`	16-bit	Bytes to transfer (`0` = 65 536 in REU hardware)

REU registers: `$DF01` command (`$B0` stash / `$B1` fetch / `$B2` swap), `$DF02–$DF03` C64 addr, `$DF04–$DF05` REU offset, `$DF06` bank, `$DF07–$DF08` length. Transfer is synchronous (CPU halted during DMA). Requires a real REU or VICE: **Settings** → **Hardware** → **RAM Expansion Module**.

Memory utilities

```
fill screen 32            # fill screen RAM $0400-$07FF with value 32 (space char)
fill color 1              # fill color RAM $D800-$DBFF with value 1 (white)

fill $0400, 1000, 32      # fill 1000 bytes starting at $0400 with value 32
fill addr, 256, 0         # addr can be word var
fill ptr, len_word, val   # all operands can be expressions / word vars

memcopy $C000, $0400, 256   # copy 256 bytes from $C000 → $0400
memcopy src_ptr, dst_ptr, 40 # word vars for source and destination

drawmem $C000, $0400, 8, 10, 40 # blit 8×10 rect from $C000 → screen at $0400,
stride 40
drawmem src_ptr, dst_ptr, w, h, 40 # word vars for src/dst
```

Both `fill` and `memcopy` support 16-bit lengths (0–65535). Use `word` variables for lengths > 255.

`drawmem src, dst, width, height, stride` copies a 2-D rectangular block. `src` is read linearly (packed rows); `dst` advances by `stride` bytes between rows — use `40` ($28) for the C64 screen or color RAM (40 columns). Width, height and stride are all 8-bit values. `src` and `dst` may be constants, `word` variables, or 8-bit expressions.

Raster IRQ

```
irq my_handler            # raster IRQ at line 0, handler = sub name or address
irq my_handler, 100       # raster IRQ at raster line 100
irq $C800, 200            # handler at fixed address
irq addr_word             # handler address from a word variable
```

Sets up a raster IRQ via the BASIC soft vector (`$0314`/`$0315`): disables CIA1 timer IRQ, ACKs pending VIC IRQ, writes raster line to `$D012`, enables VIC raster IRQ (`$D01A=$01`), writes handler address, and re-enables interrupts.

The handler **must** end with `sys $EA81` (KERNAL end-of-IRQ) — plain `RTS` or `RTI` will corrupt the stack. ACK the VIC IRQ first:

```
sub my_handler()
  poke $D019, $FF       # ACK VIC IRQ
  # ... work here ...
  sys $EA81             # JMP to KERNAL end-of-IRQ
end
```

Forward references are supported (`irq my_handler` before the sub is defined).

NMI handler

```
nmi my_nmi              # set NMI vector $0318/$0319 to handler sub or address

sub my_nmi()
  # ... NMI work here ...
  nmi_exit              # JMP $FE47 — proper NMI exit (restores A/X/Y + RTI)
end
```

`nmi handler` writes the handler address to the NMI soft vector (`$0318`/`$0319`). The hardware NMI vector at `$FFFA` points to the KERNAL NMI routine which branches through `$0318`. The handler **must** end with `nmi_exit` (emits `JMP $FE47`) — using plain `RTI` will corrupt the stack. Forward references supported.

CIA1 timer IRQ

```
cia_timer 19656, my_handler   # CIA1 timer A: fires every 19656 cycles (~50 Hz PAL)
cia_timer period, handler      # period can be a variable or expression
```

Sets up CIA1 timer A as a periodic IRQ source via the BASIC soft vector (`$0314`/`$0315`):

1. SEI — disable interrupts
2. `$DC0D = $7F` — disable all CIA1 IRQs
3. Load 16-bit period lo→`$DC04`, hi→`$DC05`
4. Write handler address to `$0314`/`$0315`
5. `$DC0D = $81` — enable CIA1 timer A IRQ
6. `$DC0E = $01` — start timer A in continuous mode
7. CLI — re-enable interrupts

The handler must end with `irq_exit` (or `sys $EA81`) and should ACK the CIA1 IRQ:

```
sub my_handler()
  poke $DC0D, $01       # ACK CIA1 timer A IRQ (read also clears it)
  # ... work here ...
  irq_exit              # JMP $EA81: restore A/X/Y + RTI
end
```

PAL timing: clock = 985 248 Hz. Period for 50 Hz = 985 248 / 50 = 19 705 cycles ≈ `$4CC9`. Forward references supported.

Error handling

```
onerr goto err_handler   # set KERNAL I/O error vector ($0300/$0301) to a label

...

label err_handler
  print "I/O ERROR"
  bye
```

`onerr goto label` writes the label address (lo, hi) to KERNAL locations `$0300` and `$0301`. When a KERNAL I/O error occurs (e.g. a failed `load` or `open`) the KERNAL executes `JMP ($0300)`, which branches to the label. Forward references (label defined after `onerr goto`) are supported.

Compile-time file embedding

```
incbin "sprites.bin"     # embed raw binary bytes at current code position
include "defs.ub"        # inline another .ub source file (lexed+parsed in place)
```

Data / Read

```
data 1, 2, 3, 255        # constant byte table
read varname             # load next byte into varname (auto-declares if needed)
```

All `data` values are collected at compile time. A 2-byte ZP pointer is automatically allocated and initialised at program start. Each `read` advances the pointer.

Inline assembly

```
sys $FFD2                # JSR $FFD2
sys $FFD2, 7             # LDA #7 ; JSR $FFD2  (pass byte value in A register)
irq_exit                 # JMP $EA81 — proper IRQ handler exit (restore A/X/Y +
RTI)
asm $EA, $EA             # inline raw bytes (NOP NOP) — legacy form
asm {
  ; Full 6502 mnemonics and addressing modes
  LDA #$07               ; immediate
  STA $0286              ; absolute
  LDA $50                ; zero-page  ($50 ≤ $FF → ZP auto-selected)
  STA $0400,X            ; absolute,X indexed
  LDA ($50),Y            ; (indirect),Y
  LDA ($50,X)            ; (indirect,X)
  JSR $FFD2              ; subroutine call
  JMP $C000              ; absolute jump
  JMP ($FFFC)            ; indirect jump

  CLC
```

```
    ADC #1
    SEC
    SBC #1

    TAX                     ; implied / transfer
    ASL A                   ; accumulator (also just: ASL)
    LSR A
    ROL
    ROR

    ; Branches — operand is an absolute address; offset is computed automatically
    BNE loop                ; forward or backward branch to local label
    BEQ done

  loop:
    NOP
  done:
    RTS

    ; #<label / #>label — lo / hi byte of a label address
    LDA #<handler
    STA $0314
    LDA #>handler
    STA $0315

    ; * — current assembly location
    JMP *

    ; Raw hex bytes (backward-compatible with old asm { $xx ... } syntax)
    $EA $EA                 ; two NOP bytes
  }
```

Addressing modes:

Syntax	Mode	Bytes	Example
(no operand)	Implied	1	`NOP`, `RTS`
`A`	Accumulator	1	`ASL A`, `LSR`
`#value`	Immediate	2	`LDA #$07`
`$zz` (0–255)	Zero-page	2	`LDA $50`
`$zz,X`	ZP,X	2	`LDA $50,X`
`$zz,Y`	ZP,Y	2	`LDX $50,Y`
`$xxxx`	Absolute	3	`LDA $0400`
`$xxxx,X`	Absolute,X	3	`LDA $0400,X`
`$xxxx,Y`	Absolute,Y	3	`LDA $0400,Y`
`($xxxx)`	Indirect	3	`JMP ($FFFC)`

Syntax	Mode	Bytes	Example
`($zz,X)`	(Indirect,X)	2	`LDA ($50,X)`
`($zz),Y`	(Indirect),Y	2	`LDA ($50),Y`
`label`	Relative	2	`BNE label` (branches only)

- `$zz` (1–2 hex digits, value ≤ 255) selects zero-page if the instruction supports it; otherwise auto-upgrades to absolute. Use `$00xx` (4 digits) to force absolute.
- Branch operands are absolute addresses; the relative byte offset is computed automatically.
- Local labels (`name:`) are scoped to the `asm { }` block. Forward branches resolved in pass 2.
- `#<label` / `#>label` yield the lo / hi byte of a label's address.
- `*` yields the current instruction address, so `JMP *` assembles as a self-loop.
- Lines starting with `$`, `%`, or a digit are emitted as raw bytes (backward-compatible).
- Inside `asm { }`, comments are `;` or `//` to end of line. (`#` is the immediate prefix, not a comment.)

Mixing `asm { }` with subroutine parameters

Parameter names are **not accessible** inside `asm { }` blocks. Use UltimateBasic statements to move values into known locations before the `asm { }` block:

```
sub set_colors(border_col, bg_col)
  poke $D020, border_col   # UltimateBasic resolves the ZP address
  poke $D021, bg_col
  asm {
    ; values are already in $D020 / $D021
    LDA $D020
  }
end
```

For routines whose entire body is assembly — especially IRQ handlers that must cross-reference each other — put all handlers in a **single top-level `asm { }` block** in the main program. Labels in the same block share scope, so `irq1` and `irq2` can reference each other freely. See `examples/raster_irq_demo.ub`.

String ↔ integer

```
numstr score, $0340      # writes "042\0" at $0340 (always 3 digits, zero-padded)
var n = str_to_int("42") # compile-time: Expr::Number(42)

print str$(score)                # print 8-bit int as 3-digit decimal string
("000"-"255")
print "Score: " + str$(score)   # usable in string concat print context
var s: string = str$(n)          # assign str$() result to a string var (shared
static buffer)
```

`str$(n)` converts an 8-bit value to a 3-character decimal string (always 3 digits with leading zeros, e.g. `5` → `"005"`, `42` → `"042"`, `255` → `"255"`) followed by a null terminator. The result pointer is stored in a permanent

ZP pair allocated at compile time.

> **Note:** `str$(n)` uses a single shared 4-byte static buffer. Calling `str$(n)` again overwrites the previous result. For concurrent display of multiple values, use `numstr` to write to separate absolute addresses.

Examples

File	Description
`examples/features.ub`	const, label/goto, poke/peek, rnd, math functions
`examples/new_features.ub`	sub params, arrays, word vars, string vars
`examples/bitmap_demo.ub`	320×200 bitmap, plot, graphics on/off
`examples/block_demo.ub`	80×50 block graphics, plot4, circle4, graphics on block
`examples/joystick_demo.ub`	joystick reading, sprite movement
`examples/mux_demo.ub`	raster sprite multiplexer (3 windows × 8 sprites = 24)
`examples/orbit_demo.ub`	24-sprite orbit with pulsating radius and random colors
`examples/plasma_demo.ub`	plasma-effect bitmap with raster bar border animation
`examples/sprite_data.ub`	sprdef shape data (included by other demos)
`examples/sprite_mux_orbit.ub`	24-sprite orbit demo with sprdef + precomputed positions
`examples/sprite_orbit_demo.ub`	8 hardware sprites in circular orbit via sin/cos table
`examples/reu_bitmap_demo.ub`	REU stash/fetch with bitmap graphics
`examples/sid_music_demo.ub`	SID music player with raster IRQ and keyboard exit
`examples/tenprint.ub`	5 TENPRINT maze implementations with menu; demos `lowercase` charset mode

Architecture

```
.ub source
  → Lexer  → Vec<Token>
  → Parser → Vec<Stmt>
  → Codegen → Vec<u8>   (raw 6502 machine code)
  → mod.rs → PRG = BASIC SYS stub + machine code
```

Two-pass compilation: Pass 1 = main code, Pass 2 = subroutine bodies (appended after main `RTS`). Forward references (`call`, `goto`, `plot`) are patched at the end.

Zero-page layout: `$02-$4F` permanent (vars, loop counters, sub params), `$50-$7F` scratch (reset per statement), `$FB` RNG seed.

CLI reference

```
ub build <input.ub> [OPTIONS]

  -o, --output <file>   Output .prg file (default: <input>.prg)
  -v, --verbose         Show zero-page layout and code hex dump
  --no-stub             Skip the BASIC SYS stub (code loads at $0801)
  --d64 [file]          Also produce a .d64 disk image;
                          without a filename defaults to <output>.d64
  --add <file>          Add an extra file to the .d64 disk image;
                          may be repeated for multiple files
  -h, --help            Show help
```

`--add` requires `--d64`. The compiled `.ub` program is always the first file on the disk; each `--add` file is appended after it. File names on the disk are derived from the source file stem, uppercased (e.g. `music.prg` → `MUSIC`).

After a successful build the compiler always prints a memory map:

```
demo.ub → demo.prg  (386 bytes)

  Load:    $080D – $0989

  Variables (zero page):
    score    ZP:$08   byte
    lives    ZP:$0A   byte
    msg      ZP:$0C   string
    ptr      ZP:$0E   word

  Subroutines:
    greet    $0900
    show     $0912

  Arrays ($C000+):
    sprites  $C000   8 bytes
```

With `-v` the output additionally shows the internal ZP allocations and a full hex dump.

Building

```
cargo build --release    # binary: target/release/ub
cargo test               # unit + integration tests
```

Known limitations

Feature	Limitation
Integer arithmetic	8-bit unsigned (0–255); `word` vars hold 16-bit values

Feature	Limitation
Subroutines	No recursion — ZP parameter slots are statically allocated
String vars	Read-only after init; assignment replaces the pointer, not the data
String concat runtime	`s1 + s2` prints sequentially — no heap allocation or length tracking
`rnd()` / `rnd(n)`	Simple LCG, not cryptographic; period = 256
`abs()` / `sgn()` / `min()` / `max()`	8-bit values only; `abs`/`sgn` treat values as signed (bit 7 = negative → `abs` two's-complements, `sgn` returns `$FF`); `min`/`max` are unsigned (0–255)
`plot`	Out-of-range pixels are silently clipped (Y ≥ 200 or X ≥ 320 → no-op)
`mplot`	No bounds checking — x must be 0–159, y must be 0–199
`plot4`	No bounds checking — x must be 0–79, y must be 0–49 (block mode)
`circle4`	Clips off-screen block pixels; useful radius is roughly 0–49 in 80×50 block mode
`chr$`	No PETSCII↔ASCII mapping — n is passed as-is to CHROUT
`music play`	Requires `load sid`; only one CIA1 wrapper is emitted (last `music play` wins)
Error reporting	Compile-time only; `onerr goto` handles KERNAL I/O errors at runtime